Worker isolation: test results (branch fix/worker-isolation, base 35884ee)

New tests: server/lib/job-isolation.test.ts

Command:

```
DATABASE_URL=postgres://stub:stub@127.0.0.1:1/stub timeout 600 npx tsx server/lib/job-isolation.test.ts
```

Result (final run, 2026-10-11, code = `f8f8384` + the integration files, which the unit file does not import): `exit=0`, `# tests 29  # pass 29  # fail 0`, duration 75.4 s (22 original tests unchanged + 7 new: Q1-Q3, M1, M2, M2b, M3). The full log is in `evidence/job-isolation.test.log`.

The test file deletes `REDIS_URL` and `MAX_CONCURRENT_SUBPROC` before its imports. This runs the child with no Redis available and the subprocess limit at its code default.

#	Test	Gap	Measured result
0	flag + PG budget partition	A	C = 4
1	child busy-loops 3 s	G	**parentMaxLag 3.0 ms**, wall 3556 ms
2	cancel	D	“Cancelled” after 2593 ms
3	revoke (ownership_lost), child ignores SIGTERM	D	“Cancelled” after 6489 ms → fenced non-retriable
4	crash holding an LLM permit	D	“Isolated job child exited unexpectedly (code=1 signal=null stage=spawn)”, retriable, permit released
5	permit brokerage	E	maxLlm 8, maxBrowser 1
6	usage forwarding	A	

#	Test	Gap	Measured result
			rows in parent buffer, child buffer 0, child PG_POOL_MAX 4
–	ProviderScoringError rebuild	D	real class
7	real 36-measure pipeline in child	G	success, 36 measures, 168 provider calls, wall 12303 ms, **parentMaxLag 67.2 ms**
A1	pool inventory	A	parentMain 4 + children 4×4 + parentUsage 2 + childUsage 0 = **22 = baseline 22**
B1	pipeline deadline, child blocked forever, 30 s grace configured	B	resolved after 2529 ms (limit 2500), “Pipeline timed out after 3s”, revoked `pipeline_timeout`, all descendants gone
B2	job deadline, child dispatching and ignoring SIGTERM	B	“Job watchdog timeout: isolated child killed at job deadline (code=null signal=SIGKILL stage=pipeline)”, non-retriable; last fixture write deadline+1 ms (tolerance 50 ms), none after
C	supervisor SIGKILL, child synchronously blocked	C	child + grandchild + detached descendant gone after 41 ms
C	supervisor SIGKILL, child idle	C	gone after 10 ms
D	retry classification table (8 rows)	D	all rows match the in-process outcome
D6	cancel in progress + job deadline	B/D	“Cancelled” after 2510 ms (killed at the deadline, not after the 30 s grace)
E1	queued IPC LLM acquire aborted while 8 held	E	pending 0 after abort, sawQueued and sawWithdrawn true
E1b	child dies with 8 held + 1 queued	E	all 8 returned, queued wait dropped
E2/ E3	child reports success but leaves a grandchild + permit alive	E	delivered after 3517 ms; at delivery child/grandchild/ proc/live = false/false/0/0; releasedWhileAlive false
E4		E	maxProc **6**

#	Test	Gap	Measured result
	3 children × 4 subprocess permits		
F1	child startup audit	F	handles `PipeWrap, Timeout` at entry and after the pipeline import (IPC pipe only); 112 modules; server modules only `storage.ts`, `db.ts`; no redis/bullmq/startup-cleanup/reconciler; pgTotal 0; usagePool false
G1	4 CPU-bound children on 2 CPUs	G	**parentMaxLag 10.4 ms**, forks 4, maxForkMs 7.6, messages 12, maxHandlerMs 0.10 (latest run: 3.9 ms, maxForkMs 3.7)
Q1	child reports success, liveness probe forced to “alive”	FIX 1	rejected `Isolated job child cleanup unconfirmed …` (non-retriable), quarantined, browser permit **not** reassigned while quarantined; released by the monitor once the probe confirms termination
Q2	probe keeps saying “alive”	FIX 1	quarantine persists; `recoverQuarantinedChildren()` returns the permits
Q3	registered group holds only an unreaped zombie (real detached process)	FIX 1	counted as terminated → clean success, no quarantine, no 2 s reap wait
M1	report → job deadline (barrier)	FIX 2	success (report had precedence)
M2	job deadline → report (barrier)	FIX 2	job watchdog timeout, non-retriable; late success discarded and logged
M2b	pipeline deadline → report (barrier)	FIX 2	pipeline timeout, retriable (existing policy); late success discarded
M3	cancel → report (barrier)	FIX 2	Cancelled; late success discarded

parentMaxLag across the three full runs on this code:

Run	Test 1	Test 7	G1	Note
run 2	3.8 ms	7.7 ms	4.5 ms	
run 3	1.6 ms	10.2 ms	4.6 ms	
run 4 (final)	3.0 ms	67.2 ms	10.4 ms	after the deadline-cuts-exit-grace change

Runs 2-4 passed 22/22 (before FIX 1/2). After FIX 1/2: run 5 failed Q3 only (the zombie fixture was not spawned `detached`, so its pgrp was wrong; fixture fixed), runs 6 and 7 passed 29/29. Latest run: test 7 parentMaxLag 5.1 ms, G1 3.9 ms. Run 1 failed only F1, because of an over-strict socket assertion; that has been fixed.

Regression suites (each run with `npx tsx <file>`, stub DATABASE_URL; all exit 0)

Suite	Result
lifecycle-fence.unit	42/42
scoring-cancellation.unit	60/60
credit-breaker	22/22
llm-usage	20/20
adaptive-concurrency	13/13
runtime-diag.unit	21/21
reconcile-resurrect	15/15
reliability	PASS
provider-resilience	112/112
transport-abort.unit	55/55

Re-run on the FIX 1/2 code (`f8f8384`), all 10 in parallel: every suite exit 0 with the counts above.

Integration suite (opt-in): server/lib/job-isolation.integration.test.ts

Command (docker `postgres:16` on 55432 and `redis:7` on 56379; schema = `drizzle-kit push` + production `initializeDatabase()`, which the test calls):

```
DATABASE_URL=postgres://ciq:ciq@127.0.0.1:55432/ciq
REDIS_URL=redis://127.0.0.1:56379 INT_REQUIRED=1 \
  timeout 420 npx tsx server/lib/job-isolation.integration.test.ts
```

Without reachable PG/Redis/Chromium it prints `SKIP` and exits 0 (exits 1 with `INT_REQUIRED=1`). Full JSON with all samples: `evidence/integration-run.json`.

What is real: `startWorker()` (BullMQ, concurrency 2, lockDuration 30 s, stalledInterval 15 s), `addBatchJobs`, `createBatchRun`/`createAnalysisJobs`, claim/complete/batch counters, `WORKER_ISOLATION_ENABLED=true` (test env only), 2 forked job children, real Chromium (`/usr/local/bin/chromium` via `fetchPdfViaBrowser` on a locally served PDF, then real `pdftotext`), real `runAnalysisPipeline(skipFetch, skipDiscovery)` on 25 seeded corpus documents per company × 6 measures, `measure_scores` written through the lifecycle fence. **What is stubbed:** LLM providers inside the child (`__setProvidersForTests`, no paid calls); discovery/fetch skipped; the test env sets `CREDIT_PAUSE_ENABLED=false` (the parent has no provider keys) and `OFFPEAK_GATE_ENABLED=false`. Each child holds 20 s after the browser fetch so the two overlap and the lock must renew.

Result (2026-10-11): `exit=0`, INTEGRATION PASS, 11/11 checks.

Check	Measured
analysis_jobs terminal state	both `completed`, attempts 1, `last_error` null
batch_runs	`completed`, completed_jobs 2, failed_jobs 0
measure_scores	6 rows per company, one attempt_id each
concurrent isolated children	2
real Chromium at peak	**18 processes**, summed RSS **2401 MB** (per-process RSS summed, shared pages double-counted, so an upper bound); all descendants 22 procs / 2801 MB
parent (supervisor) RSS	max 238 MB
BullMQ stalled events	0
lock renewals (PTTL increases on `bull:analysis:<id>:lock`)	5 and 3
descendants after completion / after stopWorker	0 / 0
orphaned puppeteer Chromium on host	none

Check	Measured
quarantined children / broker permits after	0 / llm 0, browser 0, proc 0 active
elapsed (enqueue → both terminal)	45.1 s
isolation stats	forks 2, maxForkMs 6.7, messages 374, maxHandlerMs 0.47, lateReportsDiscarded 0

Two earlier attempts failed for test-harness reasons, not product code: (1) wrong column name (`error` vs `last_error`) in the test's own query; (2) the schema lacked the raw-SQL columns that production adds in `initializeDatabase()` (`attempt_token`, `system_alerts`), so `claimJob` failed; the test now calls `initializeDatabase()`. After the aborted first run, no job children or Chromium were left behind (`pgrep` empty).

tsc

`npx tsc --noEmit -p .` gives 62 errors on the branch and 62 at HEAD `46c42ff` (changes stashed). The sorted diff of the error lists is empty, so this branch adds no new type errors. Re-run on the final code (FIX 1/2 + integration files): exit 2, still **62**, the same per-file distribution as the earlier branch run, and none in the isolation, retry-policy, fixture, child or integration-test files.

Not verified

- Unit and regression suites use a stub PG URL (port 1) and no Redis. Real Postgres/Redis/BullMQ/Chromium are covered **only** by the integration suite, on local docker, at concurrency 2. **Not** on Railway, not at production concurrency 4, not with production settings.
- Real LLM providers were not called (stubbed in the child). Discovery and live web fetch were skipped.
- Quarantine (FIX 1) was exercised with a forced probe and a real zombie, **not** with a real Chromium that refuses to die; the integration run had 0 quarantines.
- The late-report path (FIX 2) was exercised with test hooks (`setSuppressDeadlineKill`) and barriers; it did not occur in the integration run (lateReportsDiscarded 0).
- Production deadline values (pipeline 35 min, job 40 min) were not exercised. The tests use the same code paths with second-scale values.
- `WORKER_ISOLATION_ENABLED` stays **off** (the default). Staging is still needed for memory headroom (~200 MB per child, plus one Chromium per child: integration measured 18 Chromium

processes for 2 children, so up to 4 instances per replica at concurrency 4), real DB latency under the per-child cap of 4, and production CPU contention.
- No paid or real batch runs were made.